Team Discussion Exercise

Designing the Data Pipeline for Cordwell's Support Assistant

Connecting Modules 01–05 to a real-world engineering problem

How this works. Work in teams of 3–4. Pick a *scribe* and a *reporter*. You have **~45 minutes**, then the whole group reconvenes and each team reports back. This is a whiteboard-and-discussion exercise — you are *not* writing code. The goal is to reason out loud about how the tools and techniques from this week fit together on a problem that looks like the work you actually do. There is no single correct answer — defend your trade-offs.

TOOLKIT RECALL — WHAT TO DRAW ON

Before you dive in, take two minutes as a team to recall what Modules 01–05 gave you. Every prompt below should connect back to at least one of these:

- **M01 — The pipeline model:** Extract → Transform → Validate → Profile → Load; quality gates and thresholds (95% completeness, <2% duplicates, balanced categories); JSONL / Parquet / HuggingFace as LLM-ready formats; the messages-array fine-tuning format.
- **M02 — Foundations & reproducibility:** NumPy vectorization vs. Python loops, memory footprint; Jupyter for iterative work; seeds, pinned versions, "Restart & Run All," parameterized config, markdown narrative.
- **M03 — Extracting from APIs:** requests, params & auth, status codes, pagination, rate limits, exponential backoff with jitter, honoring Retry-After, defensive response validation.
- **M04 — Extracting from SQL:** SQLite vs. PostgreSQL, pd.read_sql_query, parameterized queries (never string-concatenate), chunked reads for tables larger than RAM, secure credentials.
- **M05 — Transforming & selecting:** boolean indexing, combining conditions with & | ~, .query(), chained masks, .loc for row+column selection, balancing/sampling, and handling missing values deliberately.

THE SCENARIO

Cordwell Home & Hardware is a (fictional) regional home-improvement retailer — lumber, tools, paint, appliances, garden. Its customer-support org wants an internal LLM assistant that drafts a first-pass reply for a support agent whenever a customer writes in about an order, a return, or a product question. An agent always reviews and edits before anything is sent.

Your team has been handed the data-engineering piece: assemble a clean, validated, well-balanced dataset that Cordwell can use to fine-tune (and later ground) that assistant. The raw material lives in three places:

- **A production database (PostgreSQL).** Tables include support_tickets (~6M rows: customer message, agent reply, category, resolution, timestamps, and free-text that sometimes contains names, addresses, and order numbers), orders, and products.
- **Two third-party REST APIs.** A product-reviews platform (paginated, rate-limited, token auth) and a manufacturer spec service that occasionally returns malformed or partial JSON.
- **A pile of CSV/JSON exports.** Older live-chat transcripts a previous team dumped out of a retired system — inconsistent encodings, duplicate rows, and missing fields.

Target output: messages-format JSONL, one training example per line, that a fine-tuning workflow can consume next week. Everything you build should be reproducible by a teammate on a fresh machine.

Your discussion

Move through the five parts in order — each maps to the pipeline you've been building all week. Spend the most time where your team disagrees. Capture decisions in the note lines so your reporter can speak to them.

Part 1 Frame the pipeline (Module 01)

Before any code, get the shape of the problem right.

1. Sketch Cordwell's five stages — Extract, Transform, Validate, Profile, Load. What flows into each, and what comes out the far end? What does one finished training example actually look like (which fields become the system / user / assistant messages)?
2. “Garbage in” has consequences here specifically. Name two concrete failure modes in the finished assistant that bad data at this stage could cause — and tie each to the stage that should have caught it.
3. Which of Module 01's quality thresholds (completeness, duplication, category balance) are you going to hold this dataset to, and why do those numbers matter more for a support assistant than for, say, a dashboard?

Part 2 Plan the extraction (Modules 03 & 04)

You have SQL sources and API sources. They fail in different ways and need different handling.

4. The support_tickets table is ~6M rows — larger than a laptop's RAM. How do you pull it without crashing the machine, and how would you filter to only the tickets worth training on before they ever hit memory?
5. Part of your query needs to filter by a category or date that comes from a config file. Show (in words) the safe way to parameterize that query, and explain what goes wrong if someone builds the SQL string with an f-string instead.
6. The reviews API is paginated and rate-limited. Walk through your extraction loop: how do you know when to stop, what do you do on a 429 or a 5xx, and how does honoring Retry-After / exponential backoff keep you from getting blocked?
7. The manufacturer spec API sometimes returns malformed JSON. Where does that break a naive script, and what's your defensive move so one bad response doesn't kill the whole run?

Part 3 Transform & select the training set (Module 05)

Now the pandas work: turn millions of messy rows into a smaller set of high-quality, well-chosen examples.

8. Define “a good training example” for Cordwell as a set of filter conditions (e.g., resolved tickets, a sensible reply length, a real category, not a duplicate). Express it as combined boolean conditions — and say why each parenthesized condition earns its place.
9. Different teammates own different quality rules (length, safety, category coverage). How do chained masks let you apply them step by step and log how many rows each filter removes — and why is that logging worth the extra lines?

10. Many tickets are missing a category or a resolution. What's your policy — exclude, impute, or flag — and how do you make sure missing values don't just silently vanish from your counts?
11. You only want certain columns in the final JSONL. How does .loc let you select the right rows and the right columns in one move?

Part 4 Quality, reproducibility & responsible AI (Modules 01–02)

The stages that separate a demo from something you'd trust in production.

12. You profile the data and discover 55% of tickets are “order status” questions. Why is that a problem for the fine-tuned assistant, and what would you do about it before loading?
13. support_tickets free-text contains customer names, addresses, and order numbers. Where in the pipeline do you handle that PII, and what's the risk if it rides along into the training file?
14. A teammate needs to reproduce your exact dataset next week. Which reproducibility habits from Module 02 make that possible (think seeds, pinned versions, execution order, config), and which one is easiest to forget?

Part 5 Trade-offs & curveballs (stretch)

If you have time — pick one and argue it out. These have no clean answer.

15. The reviews API allows only 30 requests/minute and you want data on 2 million products. You can't wait forever. How do you decide what's worth pulling, and what do you cut?
16. Balancing categories means throwing away thousands of “order status” examples you already cleaned. Is discarding good, clean data ever the right call? Make the case both ways.
17. Where in this pipeline would NumPy vectorization actually earn its keep — and where would reaching for it be over-engineering a step that pandas already handles fine?

Report back to the group

When we reconvene, your reporter has ~3 minutes. Come ready with:

- **One design decision** you feel good about — and the technique from this week that drove it.
- **One risk** you'd flag to Cordwell before this data ever trains a model.
- **One open question** your team couldn't settle — something you'd want to test or verify rather than assume.

Cordwell Home & Hardware is a fictional company; all data described is synthetic and for training use only.